

LC 23: Merge k Sorted Lists

Authored by Arpit Raj, LNMIIT Jaipur • July 15, 2026 • Copyright © 2026

Problem Overview

Deloitte

Samsung

Goldman Sachs

Atlassian

In a 2D vector, we are given k sorted lists and we need to merge them into one single sorted list.

Approach 1: Sequential Merge

This approach uses the solution for merging two sorted lists (LC 21) repeatedly.

- **Template:**

```
ans = first list
```

```
for every remaining list: ans = merge(ans, currlist)
```

- **Example:** If there are 4 lists (L_1, L_2, L_3, L_4), the merge sequence is:

$$L_1 + L_2$$
$$(L_1 + L_2) + L_3$$
$$(L_1 + L_2 + L_3) + L_4$$

Time Complexity: $O(K \times N)$

C++ Implementation

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if(list1==NULL) return list2;
        if(list2==NULL) return list1;

        ListNode* head;
        if(list1->val < list2->val){
            head=list1;
            list1=list1->next;
        }
        else{
            head=list2;
            list2=list2->next;
        }

        ListNode* curr=head;
        while(list1!=NULL && list2!=NULL){
            if(list1->val < list2->val){
                curr->next=list1;
                list1=list1->next;
            }
            else{
                curr->next=list2;
                list2=list2->next;
            }
            curr=curr->next;
        }

        if(list1!=NULL){
            curr->next=list1;
        }
        else{
            curr->next=list2;
        }

        return head;
    }

    ListNode* mergeKLists(vector<ListNode*>& lists) {
        if(lists.size()==0){
            return NULL;
        }

        ListNode* ans=lists[0];

```

```

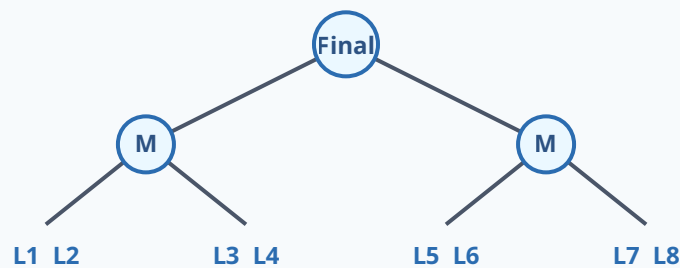
    for(int i=1; i<lists.size(); i++){
        ans = mergeTwoLists(ans, lists[i]);
    }

    return ans;
}
};

```

Approach 2: Divide and Conquer

Instead of merging sequentially, we merge pairs of lists continuously, reducing the number of lists by half each time.



- $\log_2(8) = 3$, hence completed in $\log(K)$ operations.
- For merging 2 sorted lists we just need N operations.

Time Complexity: $O(N \log K)$

Pseudocode (from notes):

```

In main function:
return solve(lists, 0, n-1);

solve(lists, int l, int r) {
    if (l == r) return lists[l];
    int mid = l + (r - l) / 2;
    ListNode* left = solve(lists, l, mid);
    ListNode* right = solve(lists, mid + 1, r);
    return merge(left, right);
}

```

C++ Implementation

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if(list1==NULL) return list2;
        if(list2==NULL) return list1;

        ListNode* head;
        if(list1->val < list2->val){
            head=list1;
            list1=list1->next;
        }
        else{
            head=list2;
            list2=list2->next;
        }

        ListNode* curr=head;
        while(list1!=NULL && list2!=NULL){
            if(list1->val < list2->val){
                curr->next=list1;
                list1=list1->next;
            }
            else{
                curr->next=list2;
                list2=list2->next;
            }
            curr=curr->next;
        }

        if(list1!=NULL){
            curr->next=list1;
        }
        else{
            curr->next=list2;
        }

        return head;
    }

    ListNode* dnc(vector<ListNode*> &lists, int l, int r){
        if(l==r){
            return lists[l];
        }

        int mid = l + (r-l)/2;

```

```

        ListNode* left = dnc(lists, l, mid);
        ListNode* right = dnc(lists, mid+1, r);

        return mergeTwoLists(left, right);
    }

    ListNode* mergeKLists(vector<ListNode*>& lists) {
        int k = lists.size();
        if(k==0){
            return NULL;
        }
        return dnc(lists, 0, k-1);
    }
};

```

Approach 3: Priority Queue (Min-Heap)

Using a Priority Queue (Min Heap) dynamically extracts the smallest node across all ***k*** lists.

- Let the lists be [1, 4, 5], [1, 3, 4], [2, 6]
- Initially, we push all **first elements** of the lists into a Min Heap. The head of our merged list will be the top element.
- After popping the smallest element, insert the **next element** from the same list into the heap (heapify).
- Continue this process until the heap is empty.

C++ Implementation


```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    struct cmp{
        bool operator()(ListNode* a, ListNode* b){
            return a->val > b->val;    // Min Heap
        }
    };

    ListNode* mergeKLists(vector<ListNode*>& lists) {
        priority_queue<ListNode*, vector<ListNode*>, cmp> pq;

        // Put first node of every list into heap
        for(int i = 0; i < lists.size(); i++){
            if(lists[i] != NULL){
                pq.push(lists[i]);
            }
        }

        if(pq.empty()) return NULL;

        // Decide head
        ListNode* head = pq.top();
        pq.pop();

        if(head->next != NULL){
            pq.push(head->next);
        }

        ListNode* curr = head;

        while(!pq.empty()){
            curr->next = pq.top();
            pq.pop();

            curr = curr->next;

            if(curr->next != NULL){
                pq.push(curr->next);
            }
        }

        return head;
    }
};

```